BENTO

Build, Evaluate, Navigate, Test & Orchestrate

Tester Watcher Setup Guide

How to deploy and configure the BENTO watcher script
on a new tester machine for automated TP compilation

Micron Technology | SSD Firmware Test Engineering
Document version: 1.2

Overview

The BENTO Tester Watcher is a Python script that runs continuously on a tester machine (Advantest). It watches a shared network folder for incoming TP packages (ZIP files) generated by BENTO on the local PC, compiles them using 'make release' (or 'make release_supermicro' for CNFG), and places the resulting TGZ file back in the shared release folder.

This guide covers:

- Prerequisites — what must exist on the tester before setup
- Verifying the watcher files on the shared folder
- Verifying the shared folder connection
- Running the watcher manually to test
- Configuring Windows Task Scheduler for automatic startup
- Adding the tester to the BENTO GUI
- Troubleshooting

Architecture

Understanding the data flow helps diagnose issues:

Local PC	Runs BENTO GUI (main.py). Creates ZIP and drops it in RAW_ZIP.
Shared Folder	P:\temp\BENTO\ — visible from both local PC and all testers.
Tester Machine	Runs watcher_main.py. Picks up ZIP, builds, copies TGZ to RELEASE_TGZ.
RAW_ZIP	P:\temp\BENTO\RAW_ZIP\ — BENTO drops ZIPs here.
RELEASE_TGZ	P:\temp\BENTO\RELEASE_TGZ\ — Tester places compiled TGZ here.

Files Reference

watcher_main.py	Entry point. Run with: python watcher_main.py --env ABIT (different ENV use SFN2 @ CNFG)
watcher_config.py	All paths and settings. Edit this for path changes.
watcher_builder.py	Runs make release via Cygwin bash. Handles memory cleanup.
watcher_copier.py	Binary copy of TGZ to shared release folder.
watcher_lock.py	Lock file and build status logic.

ENV Tokens

Each tester is identified by both its hostname AND its ENV token. The watcher only picks up ZIPs whose filename contains BOTH this tester's hostname AND its ENV token. This means two testers sharing the same ENV (e.g. two ABIT testers) will never claim each other's ZIPs.

ENV Token	Tester Hostname	Build Command
`ABIT`	IBIR-0383	make release
`SFN2`	MPT3HVM-0156	make release
`CNFG`	CTOWTST-0031	make release_supermicro

Prerequisites

Before running any steps, confirm all of the following are true on the tester machine:

Checklist

	Requirement
[ ]	Cygwin is installed (typically C:\cygwin64\)
[ ]	make, gcc, and bash are available in Cygwin (check: C:\cygwin64\bin\make.exe exists)
[ ]	The TP repo exists at C:\xi\adv_ibir_master (or your team's configured path)
[ ]	The shared folder P:\temp\BENTO is accessible from the tester (P: drive mapped)
[ ]	P:\temp\BENTO\RAW_ZIP\ and P:\temp\BENTO\RELEASE_TGZ\ folders exist
[ ]	Python is installed and accessible (run: python --version in Cygwin)
[ ]	You have VNC or remote desktop access to the tester
[ ]	You have Administrator rights (needed for Task Scheduler)

> **NOTE:** If P: drive is not mapped on the tester, map it manually: open File Explorer → This PC → Map network drive → enter \\server\temp\BENTO as the target.

Installation

STEP 1 Locate the Watcher Files on the Shared Folder

All watcher files are already available on the shared network folder. You do NOT need to copy or transfer any files manually — every tester reads from the same location.

From the tester machine, verify you can see these files at:

```
P:\temp\BENTO\watcher\
```

Expected files:

- watcher_main.py — entry point, run this to start the watcher
- watcher_config.py — all paths and settings
- watcher_builder.py — handles build execution
- watcher_copier.py — handles TGZ copy to release folder
- watcher_lock.py — manages lock and status files

> **NOTE:** If the files are not visible at P:\temp\BENTO\watcher\, check that the P: drive is mapped correctly on this tester. See the Prerequisites section for drive mapping instructions.

There is no installation required. The watcher runs directly from the shared folder. All testers always use the latest version automatically.

STEP 2 Verify Cygwin and make are Available

The BENTO watcher uses Cygwin bash to run make release. Here is why both Cygwin and Windows Command Prompt are involved:

Why Cygwin bash?	The tools used to build TP packages (make, gcc, ld) are Cygwin tools. They live in C:/cygwin64/bin/ and only work inside a Cygwin environment. Windows Command Prompt cannot find or run them.
Why not just use Cygwin?	Cygwin bash itself consumes significant RAM on top of the already heavy make release build. On tester machines with limited memory, this can cause the OS to kill the process (OOM kill) before the TGZ copy completes.
What BENTO does	BENTO calls Cygwin bash -l (login shell) to load the full Cygwin environment and PATH, runs make release inside it, then the bash process exits. The subsequent TGZ copy runs as a lightweight Python operation - much lower RAM, very unlikely to be OOM killed.

To verify Cygwin and make are correctly installed, open a Windows Command Prompt on the tester and run:

```
REM Check Cygwin bash exists:
```

```
dir C:\cygwin64\bin\bash.exe

REM Check make is available inside Cygwin:
C:\cygwin64\bin\bash.exe -lc "which make"
REM Expected: /usr/bin/make

C:\cygwin64\bin\bash.exe -lc "make --version"
REM Expected: GNU Make 4.x or similar
```

If make is not found, install it via Cygwin:

- Run C:\cygwin64\setup.exe
- Search for 'make' in the package list and select it for installation
- Also ensure these are installed: gcc-core, gcc-g++, binutils

> **NOTE:** If Cygwin is installed in a non-standard path (e.g. C:\cygwin\ instead of C:\cygwin64\), update find_cygwin_bash() in watcher_builder.py to add your path.

STEP 3 Verify the Tester Registry

Tester settings (hostname, repo path, and build command) are managed through the BENTO GUI — you do NOT need to edit watcher_config.py directly. The watcher loads its configuration automatically from P:\temp\BENTO\bento_testers.json at startup.

RAW_ZIP_FOLDER	`P:\temp\BENTO\RAW_ZIP`
RELEASE_TGZ_FOLDER	`P:\temp\BENTO\RELEASE_TGZ`
repo_dir (ABIT)	`C:\xi\adv_ibir_master`
build_cmd (ABIT)	`make release`
build_cmd (CNFG)	`make release_supermicro`
POLL_INTERVAL	`60 m (seconds)`
LOG_DIR	`C:\BENTO\logs`

> **TIP:** To verify the registry, open P:\temp\BENTO\bento_testers.json in Notepad and confirm this tester's entry is present with the correct hostname, repo_dir, and build_cmd. If it is missing or incorrect, use the + Add Tester dialog in the BENTO GUI (Step 6) to register the tester — the GUI writes the file automatically.

Running the Watcher

STEP 4 Run Manually to Test

Before setting up Task Scheduler, always run the watcher manually first to confirm it works.

Open a Windows Command Prompt on the tester and run:

```
cd P:\temp\BENTO\watcher

# For ABIT tester:
python watcher_main.py --env ABIT

# For SFN2 tester:
python watcher_main.py --env SFN2

# For CNFG tester:
python watcher_main.py --env CNFG
```

Expected startup output:

```
2026-03-12 08:17:22 | INFO     | ========================
2026-03-12 08:17:22 | INFO     | BENTO Tester Watcher  |  Environment: ABIT
2026-03-12 08:17:22 | INFO     | Hostname   : IBIR-0383
2026-03-12 08:17:22 | INFO     | RAW_ZIP    : P:\temp\BENTO\RAW_ZIP
2026-03-12 08:17:22 | INFO     | ========================
2026-03-12 08:17:22 | INFO     | Startup lock cleanup done. Removed: 0
2026-03-12 08:17:22 | INFO     | Watching P:\temp\BENTO\RAW_ZIP every 60s ...
```

Now trigger a compile from BENTO on your local PC and watch the log output. If it completes successfully, proceed to Step 5.

> **NOTE:** Press Ctrl+C to stop the watcher after testing. It does NOT need to stay running during Task Scheduler setup.

STEP 5 Configure Windows Task Scheduler

Task Scheduler will start the watcher automatically whenever the tester reboots, running it in the background as an Administrator.

1. Open Task Scheduler: Start → search 'Task Scheduler' → open
2. Click 'Create Task...' (not 'Create Basic Task') in the right panel

General Tab

- Name: BENTO Watcher ABIT (use your ENV)
- Description: BENTO tester watcher for TP package compilation
- Select: 'Run whether user is logged on or not'

- Check: 'Run with highest privileges'
- Configure for: Windows 10 or your OS version

Triggers Tab

- Click New → Begin the task: 'At startup'
- Check: Enabled
- Optional: add a 60-second delay (allows network drives to map before watcher starts)

Actions Tab

- Click New → Action: 'Start a program'

Program/script:

```
C:\cygwin64\bin\python3.exe
```

Add arguments:

```
watcher_main.py --env ABIT
```

Start in (this is required — sets working directory):

```
P:\temp\BENTO\watcher
```

Settings Tab

- Check: 'Allow task to be run on demand'
- If task is already running: 'Do not start a new instance'
- Uncheck: 'Stop task if it runs longer than' (watcher runs indefinitely)

Click OK and enter your Windows credentials when prompted.

> **TIP:** To verify the task runs: right-click it in Task Scheduler → Run. Check C:\BENTO\logs\ for the watcher log file.

Adding the Tester to BENTO GUI

STEP 6 Register Tester in BENTO

Once the watcher is running on the tester, add it to the BENTO GUI so it appears in the dropdown:

3. Open BENTO on your local PC
4. Go to the Implementation tab
5. In the 'Compile TP Package' section, click '+ Add Tester'
6. Fill in (all four fields are required):
   - Tester Hostname: the machine name (e.g. IBIR-0999)
   - Environment: select ABIT, SFN2, or CNFG
   - Repo Path: full path to the TP repo on this tester (e.g. C:\xi\adv_ibir_master) — use the 🗀 button to browse
   - Build Command: select make release or make release_supermicro (CNFG always uses make release_supermicro)
7. Click ‘Add Tester’ — the button is disabled until all three preflight checkboxes are ticked

The new tester will appear in the dropdown immediately. The registry is saved locally next to main.py and synced to P:\temp\BENTO\bento_testers.json on the shared folder so the watcher can read it. Note: restart the watcher on the tester for it to pick up the new entry.

Troubleshooting

Watcher does not pick up ZIP

ZIP hostname / ENV mismatch	ZIP filename must contain BOTH the tester hostname AND the ENV token. Format: <JIRA>_<HOSTNAME>_<ENV>_<DATE>_<TIME>.zip — e.g. TSESSD-14270_IBIR-0383_ABIT_20260312_0832.zip is only claimed by the watcher running --env ABIT on host IBIR-0383. A different ABIT tester (e.g. IBIR-0999) will not pick it up.
Watcher not running	Check Task Scheduler → BENTO Watcher task status. Logs are in C:\BENTO\logs\
P: drive not mapped	The tester cannot see RAW_ZIP. Re-map the P: drive or check network connectivity.
Pre-existing ZIP	If the watcher started AFTER the ZIP was dropped, it ignores it. Drop a new ZIP.

Build fails (make release)

make not found	Cygwin bash cannot find make. Run: C:\cygwin64\bin\bash.exe -lc 'which make'

Wrong repo path	Check repo_dir in watcher_config.py. Path must exist on the tester.
Build error	Check the build log in C:\BENTO\logs\build_<ZIP_NAME>.log for compiler errors.
OOM / killed	The watcher sleeps 10s after build for OS to reclaim RAM. If still failing, increase POST_BUILD_SLEEP_SECONDS in watcher_config.py.

TGZ not appearing in RELEASE_TGZ

Copy failed	Check watcher log for 'Binary copy failed'. Verify write permission on P:\temp\BENTO\RELEASE_TGZ
Wrong folder	TGZ is in RELEASE_TGZ\TSESSD-14270_ABIT\ subfolder, not the root.
Size mismatch	Watcher verifies copy size. If mismatch, it removes the partial file and logs an error.

Log File Locations

Watcher log	C:\BENTO\logs\watcher_ABIT_<DATE>.log
Build log	C:\BENTO\logs\build_<ZIP_NAME>.log
Status file	P:\temp\BENTO\RAW_ZIP\<ZIP_NAME>.zip.bento_status
Tester registry	Local: bento_testers.json next to main.py / Shared: P:\temp\BENTO\bento_testers.json (watcher reads from shared copy)

Contact / Escalation

If you are unable to resolve the issue after following this guide:

- Check the bento_status sidecar file — it contains the last error message from the tester
- Share the build log (C:\BENTO\logs\build_*.log) with your team lead
- Provide the watcher log for the relevant session

End of BENTO Watcher Setup Guide